

Посібник з навчання

Зберіть, запустіть детермінований процес, навчіть і отримайте готову мережу виявлення невалідного трафіку

Повний покроковий маршрут для копіювання навчальної половини конвеєра: встановіть інструментарій, зберіть платформу, запустіть детермінований процес ad-fraud, додайте власні дані й отримайте готову до розгортання багатоміткову мережу, яку споживає Посібник з розгортання.

Документ	Посібник з навчання моделі
Продукт	Jneopallium Ad-Fraud Guardian (модуль advertising-fraud)
Модель	advertising-fraud 1.0.0 · macro-F1 $\approx$ 0.97
Тренер	scripts/demo-ad-fraud/run_all.py
Вихід	Готова до розгортання конфігурація JNeopallium
Автор	Дмитро Раковський — Харків, Україна
Дата	23 червня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Чого ви досягнете
2. Де закінчується навчання і починається розгортання
3. Передумови та системні вимоги
4. Крок 1 — Отримати код і зібрати
5. Крок 2 — Запустити детермінований процес
6. Крок 3 — Зрозуміти згенеровані результати
7. Крок 4 — Етапи процесу, пояснені
8. Крок 5 — Додати власні дані (канонічна подія)
9. Крок 6 — Згенеровані артефакти: готова до розгортання мережа
10. Крок 7 — Перевірити запуск за критеріями
11. Усунення несправностей
12. Додаток — шпаргалка навчання

1 Чого ви досягнете

До кінця цього посібника ви матимете: зібрану платформу Jneorallium; повністю відтворюваний запуск процесу ad-fraud; свіжонавчену каліброву **багатоміткову модель невалідного трафіку**; і — ключовий результат — **повну, готову до розгортання мережу JНеорallium** (вісім файлів рівнів, справжні класи часу виконання, вбудовані каліброві голови й рекомендаційний запобіжник відповіді), яку напряду завантажує супровідний **Посібник з розгортання**.

Стеля безпеки для всього конвеєра

Усе, що робить навчена модель, є рекомендаційним. Вона видає кандидатні дії й докази; вона ніколи не блокує паблішера, не утримує виплату й не звинувачує людину. Звіт готовності тримає AUTOMATED_ACTION_READY хибним, доки не зафіксовано власний розмічений промисловий трафік, юридичний перегляд і процес апеляції/відкату.

2 Де закінчується навчання і починається розгортання

Фаза	Що відбувається	Документ
Навчання (цей посібник)	Зборка, запуск процесу, навчання й калібрування, видача мережі	Посібник з навчання
Розгортання	Завантаження виданої мережі, налаштування джерел подій, запуск скорера	Посібник з розгортання

Передача — це єдина тека згенерованих артефактів (Крок 6). Навчання її **створює**; розгортання її **споживає**. Тренер записує готову для JНеорallium конфігурацію рівнів і нейронів із навченими головами, калібруванням і порогами всередині, тож файли, що фіксують ваш запуск, є файлами, з яких завантажується середовище виконання.

3 Передумови та системні вимоги

Інструментарій

Інструмент	Версія	Навіщо
Java JDK	17 або новіша (LTS)	Зборка й запуск worker / скорера
Apache Maven	3.9 або новіша	Багатомодульна збірка master + worker
Python	3.10+ (тестовано 3.13)	Запускає процес; сторонні пакети не потрібні
Git	будь-яка свіжа	Клонування репозиторію

Процес працює повністю **офлайн і детерміновано** — він не просить файлів вручну. Опційні публічні джерела, недоступні, заблоковані ліцензією чи захищені обліковими даними, просто фіксуються; детермінований симулятор забезпечує покриття класів фроду, що не мають повних публічних міток.

Перевірте інструментарій

```
java -version      # очікуємо 17 або новіше
mvn -version       # очікуємо 3.9 або новіше
python --version   # очікуємо 3.10 або новіше
```

4 Крок 1 — Отримати код і зібрати

```
git clone https://github.com/rakovpublic/jneopallium.git
cd jneopallium
mvn clean install
```

Збірка встановлює `com.rakovpublic.jneopallium:worker:1.0`. Worker уже містить справжні класи нейронів, процесорів і сигналів `ad-fraud`, на які посилається навчена мережа (`com.rakovpublic.jneopallium.worker.net.neuron.impl.adfraud.*` та інші), тож для навчання не потрібні додаткові написані вручну класи.

5 Крок 2 — Запустити детермінований процес

Одна команда виконує весь процес: виявлення джерел, детермінована генерація сценаріїв, нормалізація, аудит витоку, навчання, калібрування, оцінка, експорт моделі, тести Java, відтворення демо та звіт готовності.

Linux / macOS

```
scripts/demo-ad-fraud/run_all.sh
```

Windows (PowerShell)

```
powershell -ExecutionPolicy Bypass -File scripts/demo-ad-fraud/run_all.ps1
```

Поширені прапорці

Прапорець	Ефект
<code>--offline</code>	Ніколи не намагатися завантажувати з мережі; фіксувати джерела як недоступні
<code>--quick</code>	Менший, швидший корпус (усе одно ≥ 60 подій на сценарій)
<code>--max-rows N</code>	Розмір корпусу (за замовчуванням $1900 \approx 100$ подій на сценарій)
<code>--seed N</code>	Детермінований сід (за замовчуванням 1729)
<code>--skip-java</code>	Пропустити тести Java Maven (корисно під час ітерацій)

Приклад: `python scripts/demo-ad-fraud/run_all.py --offline`. Запуск друкує зведення JSON із macro-F1, прапорцями готовності та статусом тестів Java.

6 Крок 3 — Зрозуміти згенеровані результати

Докази процесу записуються в `target/jneopallium-ad-fraud/`, а готовий до розгортання пакет моделі — у `worker/src/main/resources/model/advertising-fraud/`. Зведення має вигляд:

```
{
  "macroF1": 0.966132,
  "readiness": {
    "ENGINEERING_READY": true, "SHADOW_READY": true,
    "ADVISORY_READY": true, "AUTOMATED_ACTION_READY": false
  },
  "javaTests": "passed"
}
```

Прапорець готовності	Значення
ENGINEERING_READY	Конвеєр з'єднано, відтворювано й експортує валідний пакет
SHADOW_READY	Безпечно запускати лише для читання поруч із промислом, пишучи в приймач аудиту
ADVISORY_READY	Безпечно показувати рекомендації аналітикам
AUTOMATED_ACTION_READY	Завжди хибний , доки немає власних міток + юр. перегляду + апеляції/відкату

7 Крок 4 — Етапи процесу, пояснені

- Виявлення джерел і обмежене завантаження.** Публічні джерела (напр. атрибуція / клік-логи Criteo, краулінг `ads.txt / sellers.json`, ідентифікатори доброякісних краулерів) каталогізуються; недоступні чи захищені обліковими даними фіксуються, а не запитуються.
- Детермінована генерація сценаріїв.** Сід-симулятор виробляє реалістичні події по ~19 сценаріях (боти, клік-ферми, мотивовані когорти, `click spam/injection`, підробка постбеків, підробка інвентаря, а також законний і випадковий трафік).
- Нормалізація й аудит витоку.** Події стають канонічними рядками; аудит провалює запуск, якщо будь-який ідентифікатор пристрою / кліку / кампанії перетинає межі розбиття.
- Навчання й калібрування.** Класово-зважені, L2-регуляризовані логістичні голови припасовуються над базовими + поведінковими + нелінійними ознаками взаємодії, потім кожну голову калібрують за Platt на відкладеному розбитті калібрування.
- Пороги з урахуванням вартості й оцінка.** Робочі точки на мітку обираються за очікуваною користю; метрики включають точність/повноту/F1, PR/ROC-AUC, Brier, похибку калібрування й зважену за вартістю економію.
- Експорт, тести, готовність, документи.** Готова до розгортання мережа записується, тести Java запускаються, а звіт готовності й картки моделі/даних формуються.

8 Крок 5 — Додати власні дані (канонічна подія)

Щоб вийти за межі еталонного корпусу, ви надаєте власні канонічні події. Кожна подія зберігає й час події, й час прийому та несе поля цілісності, поведінкові, постачання й відкладеної якості, а також

метадані походження мітки. Сирі персональні ідентифікатори мають бути хешовані HMAC перед навчанням, збереженням чи логами.

Політика розбиття — це гарантія цілісності

Ніколи не розбивайте за випадковим рядком. Розбивайте за сценарієм, епізодом кампанії й детермінованим блоком реплік і тримайте ворожий холдаут — тож заявлена якість відображає справді небачений трафік. Аудит витоку забезпечує це автоматично.

Походження міток має значення: еталонна модель використовує детерміновані мітки симулятора й слабкі публічні метадані. Реальна промислова точність потребує **власного розміченого трафіку** — підтверджених чарджбеків, рішень MMP щодо фроду, ручних вердиктів аналітиків — і валідації вперед у часі та на небачених паблішерах.

9 Крок 6 — Згенеровані артефакти: готова до розгортання мережа

Кожен запуск записує теку конфігурації у стилі JNeorallium, готову до розгортання як є, — власне мережу рівнів і нейронів, з якої завантажується середовище виконання:

Артефакт	Що це
model-descriptor.json	Опис усієї мережі: 8 рівнів, 22 нейрони, мапа частот, класи часу виконання
fallback-model.json	Навчені каліброві голови + нелінійний прихований рівень (featureNames, hidden, heads)
layer-0.json ... layer-6-..., result-layer.json	Вісім файлів рівнів для розгортання
thresholds.json / calibration.json	Пороги на мітку з урахуванням вартості та Platt-калібрування
feature-schema.json / label-schema.json	Вхідні ознаки та десять міток
training-manifest.json	Джерела, кількість прикладів, політика розбиття, повні метрики
MODEL_CARD.md / DATA_CARD.md	Картки безпеки (лише рекомендації; статуси джерел)
checksums.sha256	Маніфест цілісності, який пакет Java перевіряє при завантаженні

model-descriptor.json фіксує **8-рівневу, 22-нейронну** мережу — вхід, цілісність подій, сутність/граф/якість, поведінкові докази, взаємодія ознак (нелінійний прихований рівень), навчена кореляція, рекомендаційний запобіжник і результат — де кожен нейрон посилається на конкретний клас із пакета ad-fraud worker.

Чому це важливо

Оскільки навчання видає справжню, придатну до перевірки, готову до розгортання конфігурацію з каліброваними головами й порогами всередині, між «тим, що навчили» і «тим, що запускаємо», немає кроку перекладу. Посібник з розгортання просто спрямовує середовище виконання на цю теку.

10 Крок 7 — Перевірити запуск за критеріями

1. Аудит витоку пройдено (жоден ідентифікатор не перетинає межі розбиття).
2. Macro-F1 відповідає вашій планці (еталонний запуск сягає ≈ 0.97 ; таблиця на мітку — в описі та Звіті про тестування).
3. Калібрування — справжнє Platt-масштабування на мітку (не тотожність), а похибка калібрування в межах бюджету.
4. Вісім файлів рівнів, опис і checksums.sha256 записано, і вони посилаються на очікувані класи часу виконання.
5. Тести Java проходять (javaTests: passed), а звіт готовності тримає AUTOMATED_ACTION_READY=false.

Перевірка відтворюваності

Запуск детермінований для заданого сіда. Повторний запуск тієї самої команди має відтворити той самий macro-F1 і ті самі метрики на мітку. Розбіжність сигналізує, що змінилися вхід, сід чи інструментарій.

11 Усунення несправностей

Симптом	Імовірна причина й виправлення
java -version показує < 17	Встановіть JDK 17+ і вкажіть JAVA_HOME
Процес завершується на аудиті витоку	Ідентифікатор перетинає розбиття; перевірте політику розбиття даних
Тести Java падають після зміни	Змінилися структурні лічильники; перезекспортиуйте пакет і узгодьте очікування тесту
Джерела показують dataset_unavailable	Очікувано офлайн; детермінований симулятор усе одно дає покриття
Macro-F1 нижчий за очікуваний на ваших даних	Додайте власні мітки й більше варіативності на сценарій; перевірте калібрування
Невідповідність контрольної суми в Java	Перезапустіть процес, щоб checksums.sha256 відповідав експортованим файлам

12 Додаток — шпаргалка навчання

Збірка

```
git clone https://github.com/rakovpublic/jneopallium.git && cd jneopallium && mvn clean install
```

Запуск процесу

```
# повний детермінований запуск (офлайн)
python scripts/demo-ad-fraud/run_all.py --offline
```

```
# швидка ітерація, без тестів Java
python scripts/demo-ad-fraud/run_all.py --quick --offline --skip-java

# обгортки Linux/macOS і Windows
scripts/demo-ad-fraud/run_all.sh
powershell -File scripts/demo-ad-fraud/run_all.ps1
```

*Jneorallium Ad-Fraud Guardian · Посібник з навчання. Тренер видає готову до розгортання мережу JNeorallium; як її запустити — у Посібнику з розгортання. Режим безпеки: ADVISORY / SHADOW.
Ліцензія: BSD 3-Clause.*